



HOW TO RUN IT

How to run it

What this document is

The walk from an empty folder to your first identified clip, with the real printouts and what every line means. Read PDQ_BASICS.md first if the words "hamming", "bucket" or "early exit" are new to you.

0. Set up, once

1. Install Python 3.11 or 3.12 from python.org. Tick "Add Python to PATH".
2. Open a terminal in this folder (in the file explorer, type cmd in the address bar).
3. Run:

```
pip install -r requirements.txt
```

4. Put the credentials.env file Jude sent you into this folder. Nothing in this folder can write to storage or to the database, only read.

5. Prove the setup works. This needs no key:

```
python tests/check_same_rules_as_trace.py
```

It ends with RESULT: PASS. If it does not, send the printout to Jude.

The easy way: the page

```
python app.py
```

Open <http://127.0.0.1:5077>. The Home tab says what is in place and what to do next. Films fetches the catalogue and lets you tick films (a film we do not have says so in red). Build builds. Identify takes a clip file or a link and shows the pod's printout. Wrong pulls the clips PDQ got wrong and replays any of them with one click. The rest of this document is the same road from a terminal, with the printouts explained.

1. Get the catalogue

```
python 1_get_movie_catalogue.py
```

```
listing index pieces under pdq_cache_flat_v2/shards/ ...
 5 manifests. reading them (8 at a time) ...
37 films named in the pod's index
listing fingerprint files under NEVER_DELETE_movie_hash_source_of_truth/pdq/ ...
37 fingerprint files
wrote ...data\movies_catalogue.csv (38 films)
wrote ...data\movies_to_include.txt (12 films, about 52 MB of index)
```

(This printout came from a test copy of the storage with 37 films. The real bucket holds about 9,500 films in about 630 pieces, and the listing takes a minute or two.)

What it did: read every "manifest" in the pod's index (each one names the films inside one piece of the index), listed every fingerprint file, and wrote two files.

- `data/movies_catalogue.csv` is every film. Open it in Excel. One row per film: the id, the title, the TMDB id, the size of its fingerprint file, and how big it makes the index.

- `data/movies_to_include.txt` is your starter list: 30 films picked at random. Open it in Notepad. Delete lines. Paste lines in from the catalogue. Only the number at the start of a line matters.

If you have already run step 4, the starter list begins with the films PDQ answered wrongly. Those are the ones you want in your index.

Want a fresh random pick? `python 1_get_movie_catalogue.py --new-list --seed 8`.

2. Build the index

```
python 2_build_index.py
```

```
12 films on the list
0 blobs already here, 12 to fetch
downloading: 100%|#####| 12/12 [00:04<00:00, 2.9film/s]
decoding:    100%|#####| 12/12 [00:00<00:00, 56.4film/s]
writing ...data\index ...
done in 1s: 12 films, 1,110,850 fingerprints, 0.05 GB on disk
the films inside: ...data\index\index_contents.txt
```

What it did: downloaded one fingerprint file per film into `data/blobs/` (kept, so the next build does not download again), decoded them, and wrote `data/index/`. That folder has the same layout as the pod's index, in the "flat" form a laptop can memory-map.

- `data/index/index_contents.txt` lists what is inside: one line per film, with its position number (the vidx, the number the printouts use) and its fingerprint count.

To change the index: edit the list, run step 2 again. The old index is replaced at the end.

A 2 GB index, the size that behaves like the pod's, is about 430 films:

```
python 2_build_index.py --target-gb 2
```

That picks films from the catalogue until 2 GB, rewrites the list to match, downloads about 1.4 GB of fingerprint files, and builds. Building needs about 2.5 GB of free memory while it runs. Searching needs almost none.

Have a film file on your laptop? Fingerprint it the way the pod does and add it:

```
python 2_build_index.py --from-video "D:\films\Boyhood.mp4" --title "Boyhood" --tmdb 85350
```

```
fingerprinting Boyhood.mp4 (1920x1040, 5 min). This takes about a tenth of the film's length.
 1 min of film done (6s)
...
5,236 fingerprints in 23s -> ...data\blobs\movie_900001.blob
added to movies_to_include.txt: 900001 | Boyhood (your file: Boyhood.mp4) (tmdb=85350) [0.2
    MB]
13 films on the list
...
```

Your own films get ids from 900001 up. A full film takes a few minutes.

3. Identify a clip

```
python 3_identify_clip.py clip.mp4
python 3_identify_clip.py https://www.tiktok.com/@someone/video/1234567890
```

The index it searches is named on one line near the top of 3_identify_clip.py:

```
INDEX_DIR = HERE / "data" / "index"
```

Change that line to search a different index.

A clip that is in the index

This is a 15-second clip cut from a film in the index, cropped to a phone's tall shape and re-encoded at low quality, run with --no-prescan so the rules do the work:

```
index: ...data\index (13 films, 1,116,086 fingerprints, opened in 0.09s, memory-mapped)
clip: clip_in_portrait.mp4
```

```
[ORCH][PDQ] starting query...
[ORCH][PDQ] Index ready: 1 shard(s) (13 films, 1,116,086 fingerprints, groups searched:
    ['FULL', 'PORTRAIT', 'SQUARE'])
[PROBE] likely_match=True min_neighbor_hamming=18 closest_movie=900001:Boyhood (5 min slice)
    (tmdb=85350) closest_group=PORTRAIT hashes=16 elapsed=1.12s
[PROBE] P0 triggered: min_hamming=18<30 → early exit eligible
```

```
=====
[QUERY EXIT SUMMARY]
  exit_kind=EARLY_ACCEPT
  reached_end=False
  reason=P0: probe min_hamming=18 < 30
  progress=1.12s / 15.06s
  decoded_frames=10 kept_frames=4 hashes=32
=====
```

```
Query time: 0.54s
Top candidates:
- matched= 6 avg_hamming= 31.7 offset= 130.0s movie=900001:Boyhood (5 min slice)
  (tmdb=85350)
```

```
HINTS (1 movies with avg_hamming < 45):
  1. 900001:Boyhood (5 min slice) (tmdb=85350)
    range=20-35 frames=6 avg_hamming=31.7
```

```
[EARLY EXIT] P0: probe min_hamming=18 < 30
```

```
[ORCH] PDQ early accept → movie_id=12 (P0: probe min_hamming=18 < 30)
```

```
-----
VERDICT: 900001:Boyhood (5 min slice) (tmdb=85350)
  rule: P0: probe min_hamming=18 < 30
  EARLY_ACCEPT; 0.5s
```

Line by line:

- groups searched: ['FULL', 'PORTRAIT', 'SQUARE']. The clip is tall, so PDQ searches the full-frame fingerprints, the tall crops and the square crops. A wide clip would search LANDSCAPE instead of PORTRAIT.

- [PROBE] ... min_neighbor_hamming=18 ... hashes=16 elapsed=1.12s. After 1.12 seconds of

clip, 16 fingerprints of the tall-crop family were searched once. The closest film fingerprint anywhere was 18 bits away, on the Boyhood slice.

- [PROBE] P0 triggered. One neighbour under 30 is enough for rule P0. PDQ stops here.
- exit_kind=EARLY_ACCEPT, reached_end=False, progress=1.12s / 15.06s. It stopped after

1.12 of the clip's 15 seconds. decoded_frames=10 kept_frames=4 hashes=32: ten frames were decoded, four changed enough to keep, and those four made 32 fingerprints.

- Top candidates: the scorecard. matched=6 frames landed on this film, at an average distance of 31.7, lining up at the 130-second mark of the film (the clip was cut at 2:10).

- HINTS: every film with an average under 45, best first. Here, only one.

- [EARLY_EXIT] ... and [ORCH] PDQ early accept → movie_id=12. The pod's own two lines.

movie_id=12 is the film's position in this index (line 12 of index_contents.txt), not the database id. The database id is the number before the colon in the name.

- VERDICT. This folder's one-line summary.

Notice P0 fired on one frame at distance 18 while the scorecard's average was 31.7. That is the kind of thing to look at.

The same clip, wide and low quality

```
[STREAM EARLY_EXIT] @ q_ts=0.4s - P3.5 (Stream-Only-Movie [20-30]): 8 frames @ avg_ham=28.2
(sole candidate after 0.4s)
```

P3.5 fired after 0.4 seconds because only one film in the whole index had any matched frames at all. With 13 films that happens all the time. With 9,500 films on the pod it is rare, because some film always picks up a stray frame. This is why an index near 2 GB behaves more like the pod than a small one does.

A clip that is not in the index

```
[SCAN] no early-accept -> running full pass-1 (cropped)
[PROBE] likely_match=False min_neighbor_hamming=82 closest_movie=192258:Grace (tmdb=1682658)
closest_group=FULL hashes=16 elapsed=1.25s
[REJECT] best_hamming=82>47 after 66 hashes -> no match, early reject
```

```
=====
[QUERY_EXIT_SUMMARY]
exit_kind=NO_MATCH_REJECT
reached_end=False
reason=best_hamming=82>47 after 66 hashes -> no match, early reject
progress=1.75s / 15.01s
decoded_frames=16 kept_frames=11 hashes=66
=====
```

No candidates found under max_hamming (<45).

VERDICT: no film accepted, no hints. This clip is not in your index.

The pre-scan found nothing. The probe's best neighbour was 82 bits away. After 66 fingerprints the best distance seen was still over 47, so PDQ gave up: not in the library.

Without --no-prescan

The pre-scan runs first. For a clean clip it usually ends the search on its own:

```
[SCAN] whole-clip pre-scan HIT in 3.0s: vidx=12 frames=47 min_hamming=6 -> early accept (skipped full pass-1)
```

Forty frames spread over the whole clip were hashed. Forty-seven of their fingerprints landed on film 12 under distance 28, the best at 6. That is the answer. Use `--no-prescan` when you want to watch the streaming rules decide instead.

Speed and memory

Measured on a 2.2 GB index of 505 films, on this laptop:

what	time
opening the index	under a second
a clip the streaming rules accept	1 to 5 seconds
a clip the pre-scan accepts	about 11 seconds
a clip that is not in the index	3 to 5 seconds

The program's own memory stays around 50 MB. The index pages it touches sit in Windows' file cache, and Windows takes them back the moment another program needs the memory. We checked: after a search Windows was told to take every page back, it did, and the next search still worked.

4. The job: the clips PDQ got wrong

```
python 4_pull_wrong_pdq_clips.py
```

```
wrote ...data\wrong_pdq_clips.csv    4 clips PDQ answered and a user said wrong
wrote ...data\correct_pdq_clips.csv  1 clips PDQ answered and a user said correct (the control)
of the wrong ones: 1 come with what the user said the film was; 3 answered films are in your
    catalogue
films most often given as the wrong answer:
    2 Vecina
    1 Pepe
    1 Guelwaar
```

(From a test database. The real one has a few dozen voted PDQ rows, not thousands.)

Two spreadsheets. The first is the work. The second is the control: every change to a rule has to keep these right.

The loop:

1. Run step 4. Then step 1 with `--new-list`: the starter list now begins with the films

PDQ answered wrongly. Add the films the users said were right, from the `expected_movie_id` column. Build with step 2.

2. Replay a wrong clip with step 3. Read which rule fired and what the scorecard looked like.

Do that for all of them. Write down what the wrong ones have in common. The scorecards tell you: one frame, a weak band, offsets that do not line up, one clip second.

3. Change a rule in `pdq_engine.py`. Every threshold is a named number at the top of the file with a comment. The rules themselves are in `check_early_exit`, the probe and the pre-scan are in `identify_clip`.

4. Replay both lists. Count: wrong answers removed, right answers lost. Both numbers, every time. `tests/check_same_rules_as_trace.py` will now fail on the rule you changed. That is how you know your change is the only difference from the pod.

5. When a change removes wrong answers and keeps the right ones, send Jude the two counts and the diff of `pdq_engine.py`.

Replay every clip with `--no-prescan` as well. The pre-scan hides what the rules would have done.

When something goes wrong

you see	what it means	what to do
<code>credentials.env</code> not found	Step 0.4 was skipped.	Put the file Jude sent you in this folder.
could not fetch <code>movie_123.blob</code>	That film has no fingerprint file in storage.	Take it off the list. The catalogue's <code>has_blob</code> column says which films have one.
<code>no index at ...</code>	Step 2 has not run, or <code>INDEX_DIR</code> points at the wrong place.	Run step 2, or fix the line at the top of <code>3_identify_clip.py</code> .
HTTP Error 403 on a link	The site refused the download. YouTube often does.	Save the clip another way and give the file.
MemoryError in step 2	The index is bigger than your free memory.	Build a smaller one, or close other programs.
A test prints FAIL	Something on your laptop differs from the pod.	Send the whole printout to Jude.

Words used

word	meaning
memory-map	Leave a file on disk and let Windows bring in only the parts being read.
blob	One film's fingerprint file, as stored online. About 3 MB.
vidx	A film's position number in the index. The printouts call it <code>movie_id</code> .
control	The list of things that are right today. A change must keep them right.
replay	Send a clip through step 3 again to see what the rules do with it.